

APP登录过期解决方案与操作手册

1. 问题描述

1.1 现象

APP窗口显示“登录已过期，请重新登录”，用户被迫退出当前操作，需要重新输入账号密码登录。

1.2 原因分析

项目	说明
根因	JWT Token 默认有效期 24 小时，过期后后端返回 401 状态码，前端直接跳转登录页
影响范围	APP端和WEB端均受影响
触发场景	用户登录超过 24 小时后，任何 API 请求均会触发 401 错误
用户影响	操作中断，未保存的数据可能丢失，体验差

1.3 原有流程（改造前）

用户请求API → Token过期 → 后端返回401 → 前端跳转登录页 → 用户重新输入密码

2. 解决方案

2.1 方案概述

引入**Token自动刷新机制**：当 Token 过期时，前端自动向后端发起刷新请求，在刷新窗口内（默认7天）无需用户重新输入密码即可获得新 Token 并继续操作。

2.2 改造后流程

用户请求API → Token过期 → 后端返回401
→ 前端自动调用 /auth/refresh 接口
→ 刷新成功：获取新Token，自动重试原请求，用户无感知
→ 刷新失败（超过7天窗口）：跳转登录页，提示重新登录

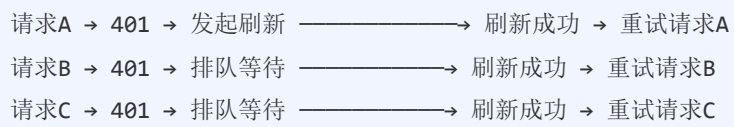
2.3 核心机制

2.3.1 时间窗口模型



2.3.2 并发请求处理

当多个请求同时收到 401 时，只发起一次刷新请求，其余请求排队等待刷新完成后自动重试：



3. 技术实现详情

3.1 后端改造

3.1.1 新增环境变量

文件: `server/.env`

```
JWT_EXPIRES_IN=24h          # Token有效期（默认24小时）
JWT_REFRESH_EXPIRES_IN=7d    # Token刷新窗口（默认7天）
```

3.1.2 新增 Token 刷新验证函数

文件: `server/src/utils/jwt.util.ts`

新增 `verifyRefreshableToken` 函数, 逻辑如下:

1. 先尝试正常验证 Token (Token可能还没过期)
2. 若 Token 过期 (`TokenExpiredError`) , 检查是否在刷新窗口内
3. 在窗口内 → 返回解码后的 Token 信息
4. 超出窗口 → 抛出"刷新窗口已过期"错误

关键代码逻辑:

```
export const verifyRefreshableToken = (token: string): any => {
  try {
    // 先尝试正常验证
    return jwt.verify(token, JWT_SECRET);
  } catch (error: any) {
    if (error.name === 'TokenExpiredError') {
      // 忽略过期, 重新解码
      const decoded = jwt.verify(token, JWT_SECRET, { ignoreExpiration: true });
      const now = Math.floor(Date.now() / 1000);
      const refreshWindow = parseDuration(JWT_REFRESH_EXPIRES_IN); // 7天=604800秒
      if (now - decoded.exp <= refreshWindow) {
        return decoded; // 在窗口内, 允许刷新
      }
      throw new Error('刷新窗口已过期, 请重新登录');
    }
    throw new Error('无效的令牌');
  }
};
```

3.1.3 新增刷新接口

文件: `server/src/modules/system/auth/auth.controller.ts`

- 请求方式: `POST /api/v1/auth/refresh`
- 无需 `authenticate` 中间件 (允许过期 Token 访问)
- 验证流程: 解析 Token → 检查用户状态 → 生成新 Token → 返回

返回数据:

```
{
  "success": true,
  "data": {
    "token": "新的JWT Token",
    "user": { "id": 1, "username": "admin", ... },
    "permissions": { ... }
  },
  "message": "令牌刷新成功"
}
```

3.1.4 注册路由

文件: `server/src/modules/system/auth/auth.routes.ts`

```
router.post('/refresh', refreshToken); // 不需要 authenticate 中间件
```

3.2 前端改造 (APP端)

文件: `client-mobile/src/api/request.ts`

3.2.1 Token 刷新状态管理

```
let isRefreshing = false // 是否正在刷新
let refreshSubscribers: Array<(token: string) => void> = [] // 排队等待的请求
```

3.2.2 刷新 Token 函数

```
async function tryRefreshToken(): Promise<string | null> {
  const currentToken = localStorage.getItem('token')
  if (!currentToken) return null
  try {
    const response = await axios.post('/api/v1/auth/refresh', {}, {
      headers: { Authorization: `Bearer ${currentToken}` },
      timeout: 10000
    })
    if (response.data?.success) {
      const newToken = response.data.data.token
      localStorage.setItem('token', newToken)
      return newToken
    }
  }
  return null
}
```

```
    } catch { return null }  
  }
```

3.2.3 响应拦截器核心逻辑

| 收到401响应
| | 是 /auth/refresh 接口本身返回401?
| | | 是 → 清除Token，跳转登录页（死循环保护）
| |
| | 当前没有正在刷新?
| | | 是 → 发起刷新
| | | | 刷新成功 → 重试所有排队请求 + 重试当前请求
| | | | 刷新失败 → 清除Token，跳转登录页
| | |
| | | 正在刷新中 → 加入排队队列，等待刷新完成后自动重试

3.3 前端改造 (WEB端)

文件: `client/src/utlis/request.ts`

与 APP 端逻辑相同，区别仅在于提示方式：

- APP端：使用 Vant 的 `showToast`
- WEB端：使用 Ant Design Vue 的 `message.error`

4. 配置参数说明

参数	环境变量	默认值	说明
Token有效期	JWT_EXPIRES_IN	24h	JWT Token 的有效时长，过期后需要刷新
刷新窗口期	JWT_REFRESH_EXPIRES_IN	7d	Token 过期后允许自动刷新的最大时长
刷新请求超时	硬编码	10000ms	刷新请求的超时时间

支持的时间单位：`s` (秒)、`m` (分)、`h` (时)、`d` (天)

调整示例：

- Token有效期改为8小时：`JWT_EXPIRES_IN=8h`
- 刷新窗口改为3天：`JWT_REFRESH_EXPIRES_IN=3d`
- Token有效期改为30分钟：`JWT_EXPIRES_IN=30m`

5. 操作手册

5.1 用户端操作

5.1.1 正常使用（无需任何操作）

系统自动在后台完成 Token 刷新，用户不会感知到任何中断。所有在刷新窗口内的操作都会自动继续。

5.1.2 超过刷新窗口后重新登录

当超过7天未使用系统时，自动刷新将失败，系统会提示"登录已过期，请重新登录"，此时需要：

- 在登录页面输入用户名和密码
- 点击登录
- 进入系统后继续操作

5.1.3 常见场景说明

场景	是否需要重新登录	说明
当天使用	否	Token在24小时内有效
隔天使用	否	Token过期但自动刷新成功
3天后使用	否	在7天刷新窗口内
8天后使用	是	超过7天刷新窗口
密码被修改	是	刷新时检查用户状态，密码变更后需重新登录

账号被禁用	是	刷新时验证用户状态
-------	---	-----------

5.2 管理员操作

5.2.1 查看当前配置

1. 登录服务器
2. 打开文件 `server/.env`
3. 查看 `JWT_EXPIRES_IN` 和 `JWT_REFRESH_EXPIRES_IN` 配置项

5.2.2 修改配置参数

1. 打开 `server/.env` 文件
2. 修改对应的参数值，例如：

```
JWT_EXPIRES_IN=12h      # 改为12小时有效期
JWT_REFRESH_EXPIRES_IN=3d # 改为3天刷新窗口
```

3. 重启后端服务：

```
# 找到后端进程
tasklist | findstr node

# 结束进程（替换PID）
taskkill /PID <进程号> /F

# 重新启动
cd d:\rubber\Seals MES System\server
npx tsx src/server.ts
```

4. 验证配置生效：修改后新登录的用户将使用新的 Token 有效期

注意：修改配置后已登录用户的 Token 不受影响，新的配置仅在下次登录或 Token 刷新时生效。

5.2.3 强制用户重新登录

如需强制所有用户重新登录（如安全审计需要）：

1. 修改 `server/.env` 中的 `JWT_SECRET` 为新值
2. 重启后端服务
3. 所有用户的 Token 将失效，需要重新登录

5.3 运维监控

5.3.1 检查刷新接口是否正常

```
# 测试刷新接口（替换为实际Token）
curl -X POST http://localhost:3000/api/v1/auth/refresh \
  -H "Authorization: Bearer <过期Token>" \
  -H "Content-Type: application/json"
```

返回结果示例：

```
// 刷新成功
{ "success": true, "data": { "token": "eyJhbG...", ... }, "message": "令牌刷新成功" }

// 超过刷新窗口
{ "success": false, "message": "刷新窗口已过期，请重新登录" }

// 无效Token
{ "success": false, "message": "无效的令牌" }
```

5.3.2 常见问题排查

问题	可能原因	解决方法
频繁提示“登录已过期”	刷新窗口设置过短	增大 <code>JWT_REFRESH_EXPIRES_IN</code> 值
刷新接口返回500	数据库连接异常	检查数据库服务状态
APP和WEB表现不一致	某端未更新代码	确认两端 <code>request.ts</code> 均包含刷新逻辑
刷新后仍提示401	<code>JWT_SECRET</code> 被修改	检查 <code>.env</code> 文件是否被改动

6. 安全说明

6.1 安全措施

1. **刷新窗口限制**：即使 Token 被窃取，超过7天后也无法刷新，必须重新认证
2. **用户状态校验**：刷新时验证用户是否仍然激活，被禁用的账号无法刷新
3. **防死循环**：refresh 接口本身的 401 不再触发刷新，避免无限循环
4. **原请求重试**：刷新成功后自动重试原始请求，用户无感知

6.2 安全建议

- 生产环境中 `JWT_SECRET` 应使用强随机字符串
- 根据业务安全要求调整刷新窗口大小
- 定期审计登录日志，关注异常刷新行为
- 如发现 Token 泄露，立即更换 `JWT_SECRET` 强制所有用户重新登录

7. 文件修改清单

文件路径	修改类型	说明
<code>server/.env</code>	新增	添加 <code>JWT_REFRESH_EXPIRES_IN=7d</code>
<code>server/src/utils/jwt.util.ts</code>	新增函数	添加 <code>verifyRefreshableToken</code> 和 <code>parseDuration</code>
<code>server/src/modules/system/auth/auth.controller.ts</code>	新增函数	添加 <code>refreshToken</code> 接口处理
<code>server/src/modules/system/auth/auth.routes.ts</code>	新增路由	添加 <code>POST /auth/refresh</code>
<code>client-mobile/src/api/request.ts</code>	重写	添加 Token 自动刷新 + 并发请求排队
<code>client/src/utils/request.ts</code>	重写	添加 Token 自动刷新 + 并发请求排队